Guion de la presentación

El mito de que los ORMs impiden la inyección SQL · máx. 10 minutos

Ponentes: Andrea Sarmiento Paredes · Iñaki Vidal Alonso

Reparto: Andrea → diapositivas 1-6 · Iñaki → diapositivas 7-11. Un solo relevo (tras la diapositiva 6). Cada ponente hace una demo.

Duración objetivo: 9:35. Hablad despacio; la demo come tiempo.

*Consejo: no leáis esto palabra por palabra. Son las ideas que hay que decir en cada diapositiva. Avanzáis con la flecha derecha; pantalla completa con la tecla **F**.*

ANDREA

1 · Portada — 25 s

«Buenas. Somos Andrea e Iñaki y os traemos *El mito de que los ORMs impiden la inyección SQL*. Vamos a explotar en vivo dos vulnerabilidades reales de TypeORM y luego las arreglamos. La idea que queremos que os llevéis: usar un ORM **no** os hace inmunes a la inyección SQL.»

2 · ¿Qué es una inyección SQL? — 55 s

«Primero lo básico. La app guarda sus datos en una base de datos y le habla con consultas SQL. Esa consulta mezcla dos cosas: las **órdenes** que escribe el programador y los **datos** que escribe el usuario. Hay inyección cuando el dato del usuario se cuela y se ejecuta como si fuera una orden. Ejemplo: un buscador por nombre. Si escribo *alice*, busca a alice. Pero si escribo `' OR '1'='1`, el filtro se rompe y devuelve a todos los usuarios. La defensa de siempre es **parametrizar**: mandar el valor por un carril aparte con una `?`, para que nunca se interprete como código. Funciona para valores… pero no siempre se puede, y ahí nacen los fallos de hoy.»

3 · ¿Qué es un ORM y por qué confiamos en él? — 55 s

«Un ORM —Object-Relational Mapping— es una librería que traduce los objetos de tu código a SQL, para no escribir las consultas a mano. TypeORM es uno de ellos. Y además parametriza solo las consultas simples, así que la gente piensa «con un ORM ya estoy a salvo» y relaja los controles. Pero el mito se rompe en tres casos: cuando la entrada va a un sitio que **no admite** la `?` —el nombre de una columna—; cuando el dato llega con una **forma inesperada** —un objeto donde esperaba texto—; o cuando hay un **bug del propio ORM**. Los dos CVE de hoy son exactamente eso.»

4 · El laboratorio — 45 s

«Para demostrarlo montamos esta app: *Campus Digital*, una intranet universitaria. A la izquierda editas tu perfil; a la derecha ves el directorio de miembros. Arranca con `npm install` y `npm start`, y en la terminal se ve cada consulta SQL en tiempo real. Dos objetivos: revelar una cuenta de administrador que está **oculta** en el directorio, y conseguir que la estudiante *alice* se convierta en admin. Cada ataque lo veréis en tres pasos: uso normal, explotación, y la defensa.»

5 · Vector 1 · CVE-2026-76848 — 60 s

«Primer fallo, en el método `distinctOn`, que sirve para agrupar resultados por una columna y no repetir filas. El problema: el nombre de una columna no se puede parametrizar con `?`, así que TypeORM lo pega **como texto** dentro de la consulta. Y aquí se lo pasamos directamente sin validar. Entonces, en vez de un nombre de columna, el atacante escribe esto: `1), * FROM user WHERE role='admin' --`. Pieza a pieza: el `1)` cierra el paréntesis que TypeORM dejó abierto; luego añade su propia consulta pidiendo los administradores; y el `--` comenta, es decir, **anula** el filtro que mantenía oculta la cuenta. En el TypeORM real esto ocurre en PostgreSQL.»

6 · Vector 1 · demo en vivo — 70 s

(Acción: enseñar la diapositiva; opcional cambiar al navegador y la terminal.)

«Lo ejecutamos. Con un valor normal como *department* saldrían los tres usuarios visibles. Pero metemos el payload en el buscador, pulsamos Filtrar… y aparece `@admin_rectorado`, la cuenta de administrador que estaba oculta. Si miráis la terminal, el SQL que generó TypeORM lleva incrustado nuestro texto: ha roto la consulta y ha anulado el filtro `visibleEnDirectorio = 1`. Hemos leído datos a los que no deberíamos tener acceso, sin concatenar SQL a mano en ningún momento. Con esto os dejo con Iñaki, que os enseña cómo se arregla.»

IÑAKI

7 · Vector 1 · solución — 55 s

«Gracias. La solución es **validar la entrada antes** de que llegue a TypeORM, con Zod. Como un nombre de columna no se puede parametrizar, usamos una **lista blanca**: solo aceptamos `id`, `username` o `department`; cualquier otra cosa se rechaza. Fijaos en el antes y el después: antes pasábamos el texto tal cual; ahora el `safeParse` de Zod corta con un 400 si el valor no está en la lista, y solo la columna ya validada llega a `distinctOn`. El mismo ataque de antes ahora devuelve *400 Bad Request* y no se ejecuta ninguna consulta.»

8 · Vector 2 · CVE-2025-60542 — 60 s

«Segundo fallo, en `save`, que guarda en la base de datos los campos del objeto que recibe. El formulario *Mi perfil* solo debería cambiar tu departamento, que es un texto. Pero si en vez de un texto mandamos un **objeto** con una clave de más —`{"department":"Robótica","role":"admin"}`—, TypeORM lo convierte en **dos cambios** a la vez. Esperaba un texto para `department`, recibe un objeto con una segunda clave `role`, y acaba escribiendo también `role = 'admin'`, que no debía tocar. Esto afecta a TypeORM anterior a la 0.3.26 con MySQL.»

9 · Vector 2 · demo en vivo — 70 s

(Acción: enseñar la diapositiva; opcional cambiar al navegador y la terminal.)

«Lo probamos. Con un texto normal, *Robótica*, solo cambiaría el departamento y alice seguiría siendo estudiante. Pero enviamos el objeto desde *Mi perfil*, pulsamos Guardar… y en el directorio alice aparece ya como **admin**. En la terminal se ve el UPDATE: en vez de tocar solo el departamento, se ha colado `role = 'admin'`. Una estudiante se acaba de dar permisos de administradora rellenando el campo del departamento.»

> **10 · Vector 2 · solución — 55 s**
>
> «Y se arregla igual, validando el tipo con Zod. Exigimos que `department` sea un **texto**: si llega un objeto, se rechaza. Añadimos `.strict()`, que prohíbe cualquier clave de más como `role`. Y además, el `id` del usuario a modificar lo tomamos de la **sesión**, no de lo que manda el cliente, para que nadie pueda editar a otro usuario. El mismo objeto malicioso de antes ahora devuelve 400, bloqueado antes de tocar la base de datos.»

> **11 · Cierre — 25 s**
>
> «Para terminar: incluso sin concatenar SQL a mano, un ORM puede ser vulnerable. El ORM protege el camino fácil, pero validar la entrada es trabajo del desarrollador. Muchas gracias, ¿tenéis alguna pregunta?»

Resumen de tiempos

#	Diapositiva	Ponente	Tiempo
1	Portada	Andrea	0:25
2	¿Qué es una inyección SQL?	Andrea	0:55
3	¿Qué es un ORM y el mito?	Andrea	0:55
4	El laboratorio	Andrea	0:45
5	Vector 1 · CVE-2026-76848	Andrea	1:00
6	Vector 1 · demo	Andrea	1:10
7	Vector 1 · solución	Iñaki	0:55
8	Vector 2 · CVE-2025-60542	Iñaki	1:00
9	Vector 2 · demo	Iñaki	1:10
10	Vector 2 · solución	Iñaki	0:55
11	Cierre	Iñaki	0:25
	Total		**9:35**

Si os preguntan (preparad estas respuestas)

- **«¿Por qué no basta el ORM?»** → Parametriza valores, pero no puede parametrizar nombres de columna ni controlar la forma del dato; ahí entra la inyección.
- **«¿Esto pasa de verdad?»** → Sí, son dos CVE publicados (2026-76848 y 2025-60542). El primero es nativo de PostgreSQL; el segundo de MySQL con TypeORM < 0.3.26.
- **«¿La solución definitiva?»** → Validar toda entrada en la frontera: lista blanca para identificadores, tipos estrictos para valores, y atar las operaciones al usuario autenticado.